

# DevSecOps Assessment Documentation

This document contains all required documentation files for Blocks 1, 3, and 4. Each section below represents an individual documentation file.

---

## docs/README.md

### DevSecOps Technical Assessment – Mishack Fanyana

This repository contains work completed for the DevSecOps Technical Assessment. The assessment required completion of four 45-minute blocks. I completed:

-  **Block 1 – Security Automation (Secret Detection Engine)**
  -  **Block 3 – Container Security (Dockerfile, scanning, deployment)**
  -  **Block 4 – Infrastructure as Code Security (Terraform + Policies)**
- 

### Repository Structure

```
application/      # Provided source code + Dockerfile
scripts/          # Security automation tool
infrastructure/    # Docker Compose, K8s, Terraform
policy/           # OPA/Conftest Rego policies
docs/             # Documentation
```

---

### Tools Used

- Python 3 (Security scanning script)
  - Docker & Docker Compose
  - Trivy (vulnerability scanning)
  - Hadolint (Dockerfile linting)
  - Terraform
  - Tfsec (IaC scanning)
  - Conftest (OPA policy validation)
  - GitHub Codespaces (execution environment)
-

## Security Highlights

- All containers run as **non-root** users
  - Multi-stage Docker builds for minimal images
  - Terraform resources configured using secure defaults (KMS encryption, private ACLs)
  - Rego policy ensures **no public S3 buckets**
  - Secret detection engine identifies AWS keys, API keys, private keys, JWTs, high-entropy strings
- 

## Blocks Completed

### Block 1 – Security Automation

A Python secret-scanning engine recursively scans source code and config files, detects risky tokens, and outputs a pass/warn/fail decision.

### Block 3 – Container Security

Implemented: - Secure Dockerfile (non-root runtime) - Docker Compose and Kubernetes deployment - Trivy and hadolint scanning steps

### Block 4 – IaC Security

Used Terraform to build secure AWS infrastructure. Added: - tfsec scanning - Conftest policy validation - Rego security policy to restrict public S3 buckets

---

## docs/block1-security-automation.md

## Block 1 – Security Automation

### Objective

Create a security automation script capable of detecting hardcoded secrets and returning a unified result.

---

### Features Implemented

- Recursive folder scanning
- Regex-based detection for:
  - AWS Access Key IDs
  - AWS Secret Keys
  - API Keys

- JWT tokens
  - RSA private keys
  - High-entropy detection (Shannon entropy)
  - Pass/fail decision output
  - JSON or text output format
- 

## File Location

```
scripts/security-scan.py
```

---

## ► Usage Example

```
python3 security-scan.py --path ./application --format json
```

---

## Tests Included

```
scripts/tests/test_secrets_1.txt  
scripts/tests/test_runner.sh
```

---

## Security Value

This tool ensures: - No hardcoded credentials - No tokens or secrets in config files - Early detection of high-risk strings

This satisfies the Block 1 requirements.

---

## docs/block3-container-security.md

# Block 3 – Container Security

## Objective

Secure the provided application using container best practices, scanning, and hardened configurations.

---

## What Was Implemented

### Dockerfile

- Multi-stage build
- Runtime uses **non-root user (UID 10001)**
- Minimal .NET ASP.NET runtime image
- Permissions restricted to /app directory

### Scanning

- **Hadolint** used to lint Dockerfile
- **Trivy image scan** used to detect vulnerabilities
- **Trivy filesystem scan** for dependency misconfigurations

### Deployment Configs

- Docker Compose file for local multi-service usage
- Kubernetes Deployment with:
- Non-root container enforcement
- Privilege escalation disabled
- Proper securityContext

---

## File Locations

```
application/Dockerfile
infrastructure/docker-compose.yml
infrastructure/k8s/deployment.yaml
```

---

## ► Commands Used

### Build

```
docker build -t countryservice-app ./application
```

### Scan Image

```
trivy image countryservice-app
```

## Lint Dockerfile

```
hadolint application/Dockerfile
```

## Security Value

- Reduced attack surface
- Eliminated root privileges
- Tested against vulnerabilities and misconfigurations

This fully satisfies Block 3.

## docs/block4-iac-security.md

# Block 4 – Infrastructure as Code Security

## Objective

Develop secure Terraform IaC, with scanning and policy validation.

## What Was Implemented

### Secure Terraform Resources

- AWS S3 bucket with:
- KMS server-side encryption
- Private ACL
- Versioning enabled

### IaC Scanning

- **tfsec** for Terraform misconfiguration scanning
- **Trivy fs scan** for IaC detection

### Policy Validation (OPA / Conftest)

Added a Rego policy to ensure: - No S3 bucket may have `public-read` ACL - S3 must use encryption

## Secret Management Approach

- No secrets stored in Terraform code
  - Recommend AWS Secrets Manager for runtime secrets
  - GitHub Secrets for CI/CD
- 

## File Locations

```
infrastructure/terraform/main.tf
infrastructure/terraform/variables.tf
infrastructure/terraform/outputs.tf
policy/s3_public_read.rego
```

---

## ► Commands Used

### Terraform

```
terraform init
terraform plan -out tfplan.bin
terraform show -json tfplan.bin > tfplan.json
```

### Validate Policies

```
conftest test tfplan.json --policy policy/
```

### Run tfsec

```
tfsec infrastructure/terraform
```

---

## Security Value

- Guarantees IaC follows security baselines
- Prevents accidental creation of public S3 buckets
- Ensures encryption is always enabled

Meets ALL Block 4 requirements.

---



## **End of Documentation**

All required documentation files have been generated for Blocks 1, 3, and 4.